

AgentOS-uCore 项目报告

project61-agentOS-happylegend

AgentOS-uCore 项目报告

1 摘要

AgentOS-uCore 在 uCore RISC-V64 教学内核上实现了一组面向 AI Agent / LLM 工作流的通用内核机制。项目的核心不是把某一个科研 Agent 平台写进内核，而是让操作系统能够识别 Agent 进程，管理 Agent Context，执行结构化工具调用，记录 Context Path，查询文件对象 metadata，处理 Agent 事件队列，维护权限、调度提示、timeline、audit ledger 和 provenance。科研平台、RUN-042 流程、Host Reader 和 LLM Relay 是演示负载和对照负载，用来展示同一工作流在普通用户态约定和 AgentOS 内核机制下的差异。

项目保留了未修改 uCore 的 baseline 目标，并在根目录提供 AgentOS-uCore 增强目标。两者运行同一科研 Agent 平台流程，输出可比较的状态文件、页面、API JSON 和图表。项目材料中的样例显示：plain target 提取 258 个状态文件，AgentOS target 提取 271 个状态文件，共有状态文件 258 个，增强目标额外提供 13 个内核证据状态文件；两者均生成 40 个 Reader 页面，AgentOS target 额外生成 13 个 API JSON；主流程记录 11 个内核参与阶段和 12 类内核事实。测试方面，项目提供 AgentOS 专项 QEMU 测试、双目标科研平台运行、Host Reader 页面检查、action runner 检查、LLM Relay 模式检查和四组对照实验。报告中的数字、图表和结论均来自项目材料。

2 1. 项目背景与目标

用户态 Agent 平台可以完成复杂工作流，但普通操作系统通常只把它看作一组普通进程和文件。工具调用、上下文、事件、权限和恢复过程都通过用户态约定表达：某个状态文件代表某个阶段，某个日志片段代表某次工具调用，某个锁文件代表某段写入保护，某个字段代表某个 Agent 的权限。这样的系统可以跑通流程，却很难回答更底层的问题：这次恢复动作是否由有权限的 Agent 发起；上下文路径是否被用户态篡改；某个文件为什么被读取；某个 Agent 为什么被唤醒；一次 LLM 调用和后续报告之间有什么因果关系。

AgentOS-uCore 的目标是把这些信息从应用约定提升到内核可管理的机制中。内核不理解某个科研项目的业务语义，也不保存云端模型密钥，更不直接访问网络。内核负责通用机制：Agent 身份、Context、结构化工具调用、文件对象查询、事件等待、权限检查、调度提示、timeline、audit 和 provenance；用户态继续负责科研 workflow、LLM prompt、报告生成、多 Agent 分工和页面展示。这种划分使系统既能服务当前科研 Agent 平台，也能在设计上扩展到代码 Agent、系统管理员 Agent、数据流水线 Agent、写作 Agent 或游戏 NPC Agent。

3 2. 总体架构

AgentOS-uCore 的总体结构可以分为七层。第一层是 Agent Process Layer, 负责 Agent 进程身份、PCB 扩展字段、role/capability、Context 映射和生命周期管理。第二层是 Agent Tool Runtime Layer, 负责结构化工具调用协议、工具表、name-based legacy ABI、id-based fast ABI、batch ‘agent_run()’、参数 key/type 校验和错误码。第三层是 Context Path Layer, 负责自动记录和手动追加上下文, 提供 query、snapshot、rollback、clear 和 detail 查询。第四层是 Agent-Friendly File/Object Query Layer, 负责真实 inode 绑定、metadata、标签、摘要、索引、查询计划、查询缓存和预取提示。第五层是 Agent Loop Layer, 负责 watch、wait、heartbeat、事件队列、多 Agent 消息、取消等待和调度原因记录。第六层是 Audit / Timeline / Provenance Layer, 把工具调用、事件、调度、预取、文件摘要和因果边统一输出。第七层是 LLM-Friendly Layer, 把 ‘llm_request’、‘llm_response’、Relay capability、事件唤醒和上下文记录纳入系统视野。

图表: AgentOS-uCore 总体架构 (文件: figures/agentos_arch.svg)

这张架构图说明了项目的机制分工。内核层只保留通用抽象, 科研平台的数据和策略保留在用户态。比如 RUN-042、lab-gene-x、stage DAG、报告写入策略和恢复策略都由用户态 orchestrator 初始化和驱动; 内核只提供 dependency record、metadata update、action commit、artifact update、event emit、timeline query 等通用能力。这样的结构避免了“为一个 demo 写内核”的问题。

4 3. 关键机制设计与实现

4.1 3.1 Agent 进程与地址空间

任务一要求支持 Agent 进程创建和地址空间设计。AgentOS-uCore 在 PCB 中增加 Agent 相关字段, 使普通进程和 Agent 进程共存。Agent 创建接口包括兼容入口 ‘agent_create()’ 和带角色模板的 ‘agent_create_role()’。角色只是 capability mask 的命名模板, 真正授权依据是内核中的 capability 字段, 而不是用户态传入的字符串或角色值。

Agent Context 映射在用户地址空间中, 当前大小为 6 页。设计上同时存在内核 shadow、用户 mirror 和用户自管 cache。shadow 是可信历史的权威来源, mirror 让 Agent 可以快速直接读取最近状态, cache 留给用户态策略保存临时信息, 不会被 ‘context_snapshot()’ 覆盖。这样做的原因是: 如果只保留用户态共享页, 性能好但可信性弱; 如果只允许 syscall 查询, 可信但热路径成本高。shadow + mirror 的组合把可信记录和快速读取分开。

‘agentfinal_ucore’ 的样例输出显示 Context size 为 24576, capacity 为 128, 并验证 ‘tamper_protected=1’、‘user_cache_preserved=1’、‘fifo_oldest=66 latest=193 dropped=65’ 等关键状态。‘agentsecurity_ucore’ 进一步验证普通进程直接调用敏感 Agent 接口会被拒绝, 低权限 Agent 伪造角色不会越权。

4.2 3.2 结构化工具调用

任务二要求 Agent 能发起结构化工具调用。AgentOS-uCore 提供两条路径: 一条是 name-based legacy ABI, 保留“工具名称 + 参数键值”的可读接口; 另一条是 id-based fast ABI, 结合 ‘agent_run()’ 批量执行最多 64 个工具操作。工具表区分 callable 工具和 syscall-only 工具, 工具参数需要通过 key/type 校验, 错误以结构化状态码返回。

这种设计服务两个目标。第一, 保留赛题语义和演示可读性: 用户可以看到 ‘echo’、‘query_file’、‘pid_info’、‘llm_request’ 这类工具名。第二, 降低热路径成本: 当 Agent 在一个推理循环中连续

执行多个工具操作时, batch ABI 可以减少 syscall 次数, 并让 Context 记录按同一 sequence 流追加。‘agentbench_ucore’ 样例中, ‘scalar_agent_run ops=256 ticks=17’, ‘batch_agent_run ops=256 ticks=5’, ‘speedup_x100=340’。这里的 tick 是同一 QEMU 环境下的相对观测, 不代表物理机绝对性能; 它说明同一批工具操作使用 batch 路径后系统调用成本下降。

4.3 3.3 Context Path

任务三要求支持 Agent 的 Context Path。AgentOS-uCore 的 Context Path 保存最近 128 条短摘要记录, 每条记录包含 tool id、状态、sequence、request id、cause/span、数值槽、tick、flags、prev hash、record hash、16 字节 payload 摘要和 16 字节 result 摘要。较长细节可通过 ‘context_detail()’ 查询。Context Path 支持自动记录工具调用, 也支持用户态手动 ‘context_push()’; 查询接口包括 ‘context_query()’、‘context_snapshot()’、‘context_rollback()’ 和 ‘context_clear()’。

Context 的关键设计不是“把所有 prompt 和 response 都塞进内核”, 而是记录足以恢复路径、审计因果和关联文件/事件的短摘要。用户态仍可以在自己的文件、报告或 Relay 中保存完整文本。内核保存的是可信路径骨架: 哪个 Agent 在什么时候调用了哪个工具, 状态是什么, 原因链是什么, 后续事件和文件证据如何连接。

图表: Context 与 timeline 查询实验 (文件: figures/experiment-context-line.svg)

这张图来自 ‘experiments/raw/context-timeline.csv’。横轴是 Context/timeline 记录数, 纵轴是用户态重建步骤或内核 snapshot/query 成本。普通路径需要从日志、状态文件和事件记录中拼接调用路径; AgentOS 通过 shadow Context、timeline cursor 和 snapshot/query 返回可信记录。记录数量增加时, 这种差异会更明显。

4.4 3.4 文件对象 metadata 查询

任务四要求支持文件相关的 Agent 能力。AgentOS-uCore 在文件对象上附加 Agent metadata, 绑定真实 ‘dev/inum’, 并通过私有 ‘agentmeta’ 文件保存持久化 metadata。metadata 模型既保留科研平台需要的 project、workflow、run_id、stage、kind、status、summary, 也扩展到更通用的 namespace、object_id、owner、type、state、priority、version、tags、label、policy tag、content digest 和 summary。查询结果返回 plan、reason、candidate count、scan count 等可解释信息。

文件查询层提供 scan 和 index 两条路径。普通用户态平台往往需要扫描状态文件或 manifest; AgentOS 可以先按 namespace、type、state 等 metadata 缩小候选集, 再读取真实文件摘要或 digest。‘agentfs_ucore’ 样例输出包括 ‘agentmeta_reload=1’、‘bulk_index scan=118 index=6 hits=1’、‘query_plan scan_plan=0 index_plan=1 reason=4 bucket=15 candidates=6’、‘digest_cache_invalidated=1’。这些输出说明文件 metadata 不是单纯展示字段, 而是参与查询计划、缓存失效和预取提示。

图表: 文件对象查询实验 (文件: figures/experiment-file-query-bar.svg)

图中横轴是文件数 32、128、512、1024, 蓝色表示普通路径触达记录数, 橙色表示 AgentOS metadata 候选数。普通路径随文件数增长而扫描更多记录; AgentOS 路径先用 metadata 缩小候选集。报告不把它描述为完整通用搜索引擎, 而描述为 uCore 根目录环境下的 Agent 友好文件对象 metadata 与内容摘要查询服务。

4.5 3.5 Agent Loop、事件与调度提示

任务五要求支持 Agent Loop。AgentOS-uCore 为每个 Agent 提供 16 槽 FIFO 事件队列、最多 8 项 watch 条件、‘agent_wait()’、‘agent_unwatch()’、‘agent_wait_cancel()’、‘agent_heartbeat()’ 和 ‘agent_heartbeat_stop()’。有限 timeout 会进入睡眠状态，由事件入队、heartbeat 到期或 deadline 到期唤醒，不通过反复 ‘yield()’ 轮询。事件记录包含 cause/span，便于把消息、工具调用和后续处理连接起来。

‘agentloop_ucore’ 样例输出包括 ‘fifo=1’、‘event_causality=1’、‘overflow_dropped=1’、‘unwatch=1’、‘timeout_sleep_no_poll=1’、‘timer_unwatch=1’、‘heartbeat_wake_stop=1’ 和 ‘wait_cancel=1’。‘agentsched_ucore’ 进一步验证 Agent 感知调度：不同角色模板有不同权重，orchestrator 可以配置目标 Agent 的 weight、priority 和 budget，调度记录包含 reason、score、事件数量和虚拟运行量。

图表：事件等待实验（文件：figures/experiment-event-box.svg）

这张箱形图比较事件数量变化时普通用户态轮询和 AgentOS wait/wake 的差异。它的意义不在于某一次 tick 多小，而在于展示事件增加后，普通路径需要更多轮询，而 AgentOS 路径由内核事件队列和睡眠唤醒机制处理。

4.6 3.6 Audit、Timeline 与 Provenance

Agent 工作流需要的不只是“能运行”，还要能解释。AgentOS-uCore 把 Context、事件、调度、预取、文件摘要和 Agent 行为写入统一短记录。‘agent_timeline_snapshot()’ 输出统一 timeline；‘agent_timeline_query()’ 支持按 source、tick、span、kind、pid、tool、event、status、flags 和 cursor 查询；‘agent_timeline_wait()’ 和 ‘agent_timeline_read()’ 支持等待某类记录出现后再读取。‘agent_audit_snapshot()’ 面向 orchestrator 导出系统级短记录，记录包含 prev hash 和 record hash。‘agent_provenance_snapshot()’ 输出 Context、Audit、Prefetch 等节点之间的因果边。

图表：AgentOS 运行事实管线（文件：figures/agentos-telemetry-pipeline.svg）

这部分是项目区别于普通用户态平台的重要地方。普通平台即使保存日志，日志仍然是事后约定；AgentOS 把 Agent 的关键行为纳入内核记录，从工具调用、事件消费到调度原因和文件预取都能在统一 timeline 中被查询。‘labdemo_ucore’ 样例输出包含 ‘global_audit=1’、‘unified_timeline records=... context=1 event=1 sched=1 prefetch=1 digest=1’、‘provenance_graph edges=... message=1 prefetch=1 digest=1’，说明多 Agent 协作中的消息、预取和文件摘要能连成因果图。

4.7 3.7 LLM-friendly 路径

项目把 LLM 视为 Agent 工作流中的一等对象，但不让内核直接访问云端模型。内核提供 ‘llm_request’、‘llm_response’、Relay capability、request id、span id、quota、timeout、status、Context 记录和 LLM_DONE 事件。用户态或宿主机 Relay 负责 template/offline 模式和真实 cloud 模式。这样既能让 LLM 调用进入 audit、timeline 和 provenance，又避免内核保存 API key、实现 HTTP/TLS 或依赖某个云端 endpoint。

‘agentllm_ucore’ 样例输出包含 ‘relay_timeline=1’、‘requester_done=1’、‘template_relay=1’。README 还说明宿主机 Host LLM Relay 支持 template 模式和 cloud 模式；cloud 模式可由本机配置 DeepSeek/OpenAI-compatible key 后生成 review summary、method check、recovery note、writer summary、项目评审意见和最终报告摘要。公开仓库默认仍能用 template 模式复现，不需要提交任何密钥。

5 4. 科研 Agent 平台作为演示负载

科研 Agent 平台是项目的主要演示负载。baseline target 在未修改 uCore 上运行同一平台，状态靠普通用户态程序、文件和约定维护；AgentOS-uCore target 运行同一业务流程，但关键阶段使用内核 Agent 服务。增强目标入口是 ‘rp_agentos_orch’，它创建 orchestrator Agent，初始化 ‘rp_agentos_mainflow’，再运行完整科研平台流程。主流程追加 11 个内核参与阶段，并覆盖 12 类内核事实：可信 Context、通用依赖图与依赖驱动预取、metadata 索引查询、Agent 事件通知、通用 action/artifact 恢复、ledger/provenance 观察、sentinel 权限拒绝、timeline 观察、文件编辑租约、workbench 文件校验、证据包 provenance 和真实任务报告/audit Context。

这套演示负载的价值在于对照。同一个科研流程在 plain target 中可以完成，说明项目没有把功能偷换成只在增强内核中可运行；同一个流程在 AgentOS target 中额外输出内核证据，说明内核机制进入了主流程，而不是旁边单独跑一组测试。README 样例中 ‘dual_platform_state_compare’ 显示 plain 状态文件 258 个、AgentOS 状态文件 271 个、共有 258 个、增强目标额外 13 个；‘run_result_match=1’ 表明两个目标的运行结果可比较；‘agentos_evidence_checks=32’、‘agentos_mainflow_stages=11’、‘agentos_mainflow_facts=12’ 说明增强目标的内核证据被脚本检查。

Host Reader 把 ‘rp_*’ 状态文件渲染成浏览器页面。两个目标都生成 40 个页面，AgentOS target 额外提供内核输出文件和 API JSON。演示时可以先运行 ‘make dual-platform-run TOOLPREFIX=riscv64-linux-gnu-’ 生成结果，再运行 ‘make demo-reader’ 打开本地页面。页面用于展示双目标概览、Run、Workflow、Workbench、Project、Review、Delivery、LLM 和 Compare，不属于内核机制本身，而是评审阅读和视频展示入口。

6 5. 测试与性能观测

项目的测试分为三类。第一类是 AgentOS 内核专项 QEMU 测试，脚本依次运行 ‘agentfinal_ucore’、‘agentfs_ucore’、‘agentscan_ucore’、‘agentloop_ucore’、‘agentsched_ucore’、‘agentconflict_ucore’、‘agentllm_ucore’、‘agentbench_ucore’、‘labbench_ucore’、‘labdemo_ucore’ 和 ‘agentsecurity_ucore’，期望每个程序输出 ‘parent passed’。第二类是双目标科研平台运行，比较 baseline 和 AgentOS 两个目标的状态文件、页面、API JSON 和内核证据。第三类是宿主机工具检查，包括 Host Reader、action runner、LLM Relay 模式、图表数据契约和 SVG 布局检查。

图表：双目标运行观测面板（文件：figures/runtime-observation.svg）

运行观测面板把阶段执行、状态产物、内核证据、Agent 启动方式和 QEMU 健康状态放在同一画面。它不是一个额外结论，而是告诉读者本次数据是否可用：脚本是否跑完、两个目标是否都有状态文件、增强目标是否多出内核证据、QEMU 是否出现超时或长时间无输出。

图表：用户态成本项与 AgentOS 替代机制（文件：figures/cost-replacement.svg）

成本替代图把普通用户态路径的问题和 AgentOS 机制逐项对齐。左侧是普通平台中需要承担的成本，例如重建上下文、扫描状态文件、用约定字段表达权限、用锁文件避免并发写入、轮询事件；右侧是 AgentOS 实际使用的替代机制，例如内核 Context Path、metadata 索引、capability 检查、事件队列、文件编辑租约、timeline、audit 和 provenance。这张图用于解释“为什么要把 Agent 能力放入内核”。

图表：Runner Tick 对照（文件：figures/runner-ticks.svg）

Runner tick 图读取 ‘rp_backend_exec’ 中的 ‘runner_case’ tick 字段。plain target 的用户态路径会记录上下文重建、manifest 扫描、文件事件交接和追加日志等动作；AgentOS target 对应路径会记录 Context snapshot、metadata index、event queue 和 ledger snapshot 等内核辅助动作。因为运行环境是 QEMU，tick 只作为同一输入下的相对观测。

图表：Runner 成组场景相对倍数（文件：figures/runner-speedup.svg）

Runner speedup 图由 ‘runner-sweep.csv’ 生成。CSV 保留每个场景的 plain case、AgentOS case、两边 tick、节省 tick 和相对倍数。它延续参考项目“成组场景 + 机制解释”的测试写法，让读者看到不同类别工作项从用户态约定迁移到 AgentOS 机制后的相对成本变化。

项目还设计了四组更贴近 Agent 工作流本质的对照实验。第一组改变文件数 32、128、512、1024，比较普通扫描记录数与 AgentOS metadata 候选数。第二组改变 Context/timeline 记录数 128、512、2048、8192，比较用户态重建步骤与内核 snapshot/query 成本。第三组改变事件数 8、32、128、512，比较用户态轮询次数与内核 wait/wake 次数。第四组改变并发 Agent 数 2、4、8、16，比较用户态锁文件/覆盖风险与 AgentOS lease/capability 拒绝效果。

图表：并发 Agent 写入实验（文件：figures/experiment-concurrency-heatmap.svg）

并发实验的重点是“被拒绝”不等于失败。普通用户态路径在 Agent 数增加时依赖锁文件和约定字段，仍然存在覆盖风险；AgentOS 在内核检查 lease 和 capability，非法写入会被拒绝并记录。热力图展示的是残余写入风险，而原始 CSV 保留拒绝次数。这个实验说明权限和租约机制能把错误写入挡在内核路径上。

图表：四组实验综合观测（文件：figures/experiment-monitor-area.svg）

综合观测图把四组实验的 plain 中位数减 AgentOS 中位数形成的节省操作数放在同一张面积图里。它不试图用一个数字概括所有性能，而是分维度展示减少扫描、减少重建、减少轮询和降低冲突风险。

7 6. 赛题任务对应关系

这张表的重点是把任务要求落到具体机制和测试入口上。任务六不是通过内核硬编码科研业务完成，而是通过用户态科研平台调用通用 AgentOS ABI 完成。这样可以同时满足赛题演示要求和通用内核设计要求。

8 7. 可复现运行方式

项目面向 Windows 用户时建议使用 WSL2 Ubuntu 环境。需要的主要依赖包括 ‘git’、‘make’、RISC-V 交叉工具链 ‘riscv64-linux-gnu-’、‘qemu-system-riscv64’ 和 Python 3。仓库提供 baseline 和 AgentOS 两个目标，普通用户只需要在项目根目录运行命令。

双目标演示命令：

```
make dual-platform-run TOOLPREFIX=riscv64-linux-gnu-
make demo-reader
```

第一条命令运行普通 uCore 和 AgentOS-uCore 两个目标，提取状态文件并生成 ‘results/latest/’ 下的汇总材料。第二条命令启动本地 Reader 服务，默认页面位于 ‘http://127.0.0.1:8767/’。完整验证命令：

```
make full-verify TOOLPREFIX=riscv64-linux-gnu-
```

AgentOS 专项测试也可以单独运行：

```
TOOLPREFIX=riscv64-linux-gnu- QEMU=qemu-system-riscv64 CASE_TIMEOUT=240s bash scripts/run-agent-
```

项目文档说明，‘results/latest/’ 会包含 ‘summary.csv’、‘runner-sweep.csv’、四组实验 raw CSV、‘experiment-stats.csv’、‘mechanism-notes.csv’、多个 HTML 页面、报告 Markdown 和图表 SVG。该目录适合材料复查和答辩时查看，但默认被 ‘gitignore’ 忽略，实际数值以本机最新运行结果为准。

9 8. 工程口径与当前限制

第一，报告中的 tick 只用于同一 QEMU、同一输入、同一脚本下的相对观测，不用于宣称物理机绝对吞吐。第二，文件对象查询当前以 uCore 简化根目录和固定 metadata 容量为基础，不等同于完整多级目录全文检索系统。第三，Context Path 保留最近 128 条短摘要和 detail 查询，不保存完整 prompt/response 原文。第四，audit ledger 和 provenance 当前是内存中的轻量事实链，不宣称达到生产级持久化签名审计。第五，云端 LLM 调用由用户态或宿主机 Relay 完成，内核只管理结构化请求、响应、事件和审计记录。

这些限制不削弱项目主张，反而明确了它的范围：AgentOS-uCore 的贡献是把 Agent 工作流的关键系统事实纳入内核机制，而不是把教学内核扩展成完整生产操作系统或云端 AI 服务。

10 9. 总结

AgentOS-uCore 展示了一条清晰路线：AI Agent 不必只停留在用户态脚本和状态文件约定中。通过 Agent 进程身份、Agent Context、结构化工具调用、Context Path、文件对象 metadata 查询、事件驱动 Agent Loop、权限与租约、timeline、audit 和 provenance，操作系统可以为 Agent 工作流提供更可信、更可观察、更可约束的执行基础。科研 Agent 平台证明这些机制可以进入真实多 Agent 流程；baseline/AgentOS 双目标对照和四组实验图表则说明，同一负载下内核机制能够减少扫描、减少重建、减少轮询并降低并发写入风险。

最终项目可以概括为：在 uCore 中实现面向 AI Agent / LLM 工作流的通用内核支持层，并用科研 Agent 平台作为主要演示负载，展示这些机制如何服务真实工作流。内核不保存密钥，不硬编码科研业务，不直接调用云端模型；它管理的是 Agent 工作流中那些本应由操作系统理解的身份、上下文、工具、文件、事件、权限和因果事实。